ÇANKAYA UNIVERSITY
Department of Computer Engineering

CENG 471 — Parallel Computing

Term Project Report

Parallel Traffic Flow Optimizer

Student Name	Student ID
Mete Serpil	202211061
Nurettin Efe Alver	202211007
Arda Gönül	202211027

Submission Date

April 27, 2026

Table of Contents

1 Introduction3

2 Problem Description4

2.1 Traffic Network Model4

2.2 The Three Optimization Sub-Problems4

2.3 Limitations of the Serial Approach5

3 Parallel Design5

3.1 Decomposition Strategy5

3.2 Thread Worker Architecture6

3.3 Synchronization Mechanisms6

4 Algorithms7

4.1 Signal Optimization — Webster's Formula7

4.2 Congestion Detection8

4.3 Route Computation — Dijkstra's Algorithm8

5 Code Listings9

6 Results and Screenshots10

7 Discussion11

7.1 Amdahl's Law Analysis11

7.2 The Python GIL and Its Impact12

7.3 Bottlenecks Observed13

8 Conclusion13

9 References14

1. Introduction

Traffic flow optimization is a critical challenge in modern urban planning and transportation engineering. As cities grow and vehicle density increases, the ability to intelligently manage traffic signals, detect congestion, and route vehicles efficiently becomes essential to reducing commute times, minimizing fuel consumption, and lowering carbon emissions. Suboptimal traffic signal timing alone is estimated to account for a significant proportion of urban delay, with studies indicating that coordinated signal control can reduce average vehicle delay by up to 20–40% in dense road networks.

The computational demands of traffic optimization present a natural opportunity for parallelism. A real city may contain thousands of intersections, hundreds of thousands of road segments, and millions of vehicle journeys per day. Optimizing such a network in real time using a strictly sequential algorithm is infeasible at scale. The individual optimization tasks — computing the optimal signal timing for each intersection, evaluating the congestion ratio on each road, and computing the shortest route for each vehicle — are largely independent of one another and thus amenable to concurrent execution across multiple threads.

This project implements a **Parallel Traffic Flow Optimizer** using Python's threading module, which is the CPython language runtime's direct wrapper around the operating system's native POSIX thread (Pthreads) API. The system models a city as an N×N grid of intersections connected by bidirectional road segments. A configurable number of vehicles are generated and assigned source–destination pairs. Three optimization phases are then executed in parallel: signal timing optimization using Webster's formula, congestion detection via flow-to-capacity ratio analysis, and shortest-path route computation using a congestion-weighted variant of Dijkstra's algorithm.

A serial (single-threaded) baseline implementation of identical algorithms is also provided, enabling rigorous performance comparison. Benchmark results are collected across varying network sizes and thread counts, and speedup and efficiency metrics are computed and visualized.

Work Division

Student	Student ID	Responsibilities
Mete Serpil	202211061	Network data structures (network.py), serial implementation (serial_optimizer.py), benchmarking (benchmark.py)
Nurettin Efe Alver	202211007	Parallel implementation (parallel_optimizer.py), thread worker design, synchronization primitives
Arda Gönül	202211027	Core algorithms (algorithms.py), visualizations (visualize.py), report generation

2. Problem Description

2.1 Traffic Network Model

The traffic network is represented as a directed graph G = (V, E), where V is the set of intersections and E is the set of directed road segments. For this project, the graph is structured as an N×N grid, yielding $|V| = N^2$ intersections and approximately $2 \times N \times (N-1)$ bidirectional road segments. Each intersection $i \in V$ is characterized by the following attributes:

- **vehicle_count**: the number of vehicles currently queued at the intersection.
- **green_time**: the current green phase duration in seconds.
- **red_time**: the current red phase duration in seconds.
- **optimized_green**: the green phase duration computed by the optimizer.
- **wait_time**: the estimated average vehicle wait time after optimization.

Each road segment $r \in E$ connecting intersection u to intersection v is characterized by:

- **capacity**: the maximum number of vehicles the segment can sustain.
- **flow**: the current number of vehicles occupying the segment.
- **congestion**: the ratio flow / capacity $\in [0, 1]$.
- **travel_time**: the base free-flow travel time in seconds.

Vehicles are modelled as simple agents with a source intersection and a destination intersection. Each vehicle requires a route — an ordered sequence of intersection identifiers — computed by the route optimization phase.

2.2 The Three Optimization Sub-Problems

The full optimization pipeline is decomposed into three distinct, largely independent sub-problems, each of which constitutes a phase of execution:

- **Signal Optimization**: For each intersection, compute the optimal green phase duration based on the current vehicle demand. This phase does not depend on road congestion values and can be computed entirely independently per intersection.
- **Congestion Detection**: For each road segment, compute the congestion ratio and flag the segment as a bottleneck if the ratio exceeds a defined threshold. This phase is likewise embarrassingly parallel at the road-segment level.
- **Route Computation**: For each vehicle, compute the minimum-cost path from source to destination on the current network graph, where edge costs are modulated by congestion. Vehicle routes are independent of one another, making this phase parallelizable per vehicle.

2.3 Limitations of the Serial Approach

In a serial implementation, all three phases are executed sequentially over all network elements in a single thread. For a network of N^2 intersections, R road segments, and V vehicles, the serial time complexity is approximately:

```
T_serial = O(N^2) + O(R) + O(V * (R + N^2 * log N^2))
```

Equation 1: Serial time complexity (signal optimization + congestion detection + Dijkstra routing)

The Dijkstra component dominates for large networks. As network size and vehicle count scale up, the serial approach rapidly becomes a bottleneck. A network of 100 intersections, 180 road segments, and 500 vehicles already requires several seconds of compute time per optimization cycle, which is unacceptable for real-time adaptive traffic control systems. Parallelism is therefore not merely an academic exercise but a practical engineering necessity.

3. Parallel Design

3.1 Decomposition Strategy

The parallel design follows a **data parallelism** model. The collection of work items for each phase — intersections, road segments, or vehicles — is partitioned into T approximately equal-sized subsets, where T is the number of worker threads. Each thread processes its assigned subset independently. This approach is appropriate because the computation on each work item is independent, with the sole shared state being the results collection and the global network object (which threads read from but write to only through per-object locks).

Partitioning is performed by the following scheme: for a list of N items and T threads, thread i receives items in the index range $[i \times \lfloor N/T \rfloor, (i+1) \times \lfloor N/T \rfloor)$, with the final thread absorbing any remainder items. This guarantees load balance when $N \gg T$.

```
chunk = N // T thread_i receives items[i*chunk : (i+1)*chunk]
```

Equation 2: Workload partitioning scheme

3.2 Thread Worker Architecture

Three distinct thread worker classes are defined, each inheriting from threading.Thread:

SignalOptimizer

Each SignalOptimizer thread receives a partition of Intersection objects. It applies Webster's formula (see Section 4.1) to compute the optimal green time for each intersection in its partition. Results are written back to the intersection's optimized_green and wait_time fields under the intersection's per-object lock. A threading.Barrier is used to ensure all signal threads have completed their local computation before any thread writes to the shared global results dictionary.

CongestionDetector

Each CongestionDetector thread receives a partition of Road objects. It computes the congestion ratio for each road in its partition and identifies bottleneck segments. Bottleneck records are appended to a shared list, which is protected by a global threading.Lock acquired only at the moment of append, minimizing contention.

RouteFlowComputer

Each RouteFlowComputer thread receives a partition of Vehicle objects. It runs Dijkstra's algorithm (see Section 4.3) for each vehicle in its partition, computes the optimal route, and then increments the flow counter on each road along that route. Road flow updates are performed under the per-road lock to prevent race conditions. Aggregate statistics (total wait time, vehicles routed) are communicated to the main thread via a queue.Queue instance.

3.3 Synchronization Mechanisms

The following table summarizes all synchronization primitives employed in the parallel implementation and the rationale for each choice:

Primitive	Location	Purpose
threading.Lock (per-object)	Each Intersection & Road	Protect mutable fields during concurrent writes
threading.Lock (global)	Shared results dict / bottleneck list	Aggregate thread-local results safely
threading.Barrier	SignalOptimizer threads	Phase synchronization before result aggregation
queue.Queue	RouteFlowComputer threads	Thread-safe statistics collection without explicit locking

The use of fine-grained per-object locks (one lock per Intersection, one per Road) is a deliberate design choice to minimize lock contention. A single global lock protecting all network state would serialize threads unnecessarily. The per-object approach allows threads to operate concurrently on disjoint portions of the network.

4. Algorithms

4.1 Signal Optimization — Webster's Formula

The optimal signal cycle length computation is based on Webster's 1958 formula [1], a foundational result in traffic engineering. Webster derived the cycle length C that minimizes average vehicle delay at an isolated intersection:

```
C = (1.5 x L + 5) / (1 - Y)
```

Equation 3: Webster's optimal cycle length formula

Where:

- **C** is the optimal cycle length in seconds.
- **L** is the total lost time per cycle, set to 4 seconds (a standard empirical value).
- **Y** is the sum of critical lane volume-to-saturation flow ratios, approximated here as vehicle_count / 100, clamped to a maximum of 0.90 to prevent cycle length divergence.

The cycle length is subsequently clamped to the range [40, 120] seconds to reflect practical constraints on signal hardware. The green time allocated to the primary phase is then:

```
g = clamp( C x (vehicle_count / total_demand),  10,  80 )
```

Equation 4: Optimal green phase duration

The estimated average vehicle wait time at the intersection is computed as:

```
W = (C - g) x (1 + Y)
```

Equation 5: Estimated average wait time

This formula yields shorter green phases for lightly loaded intersections and longer green phases for heavily loaded ones, directly reducing queue lengths and average delay. Each intersection's computation is fully independent, making this phase embarrassingly parallel.

4.2 Congestion Detection

The congestion level of a road segment is quantified by the volume-to-capacity ratio (v/c ratio), a standard measure in the Highway Capacity Manual:

```
congestion(r) = flow(r) / capacity(r),   congestion in [0, 1]
```

Equation 6: Congestion ratio definition

A road segment r is classified as a **bottleneck** if its congestion ratio meets or exceeds a defined threshold of 0.80, consistent with Level of Service (LOS) D thresholds in standard traffic engineering practice. A ratio of 1.0 indicates the road is at full capacity, while ratios exceeding 1.0 are physically impossible in this model (flows are constrained during simulation).

Bottleneck identification has direct operational significance: the optimizer can alert traffic management centres to congested corridors, enabling rerouting recommendations or incident response dispatch. The detection computation per road is O(1) and fully independent across segments.

4.3 Route Computation — Dijkstra's Algorithm

Shortest-path computation for each vehicle uses Dijkstra's algorithm [2], adapted to account for current traffic conditions. The weight of a road segment r(u, v) is defined as:

```
w(u, v) = travel_time(u,v) x (1 + congestion(u,v))
```

Equation 7: Congestion-weighted edge cost

This formulation penalizes routes that traverse congested roads, naturally distributing traffic load across the network and avoiding further overloading of bottleneck segments. When congestion is zero (free-flow conditions), the weight reduces to the free-flow travel time. When congestion is 1.0 (full capacity), the effective travel time is doubled.

Dijkstra's algorithm is implemented using a binary min-heap priority queue (Python's heapq module). For a graph with |V| vertices and |E| edges, the time complexity is O((|V| + |E|) log |V|). For each vehicle, the algorithm is run independently on the current network state (with congestion values fixed at the start of the routing phase), enabling concurrent execution across threads without network state modification conflicts.

After route computation, each vehicle's assigned route is used to update the flow counts on the traversed road segments. These updates are performed atomically under per-road locks to maintain data integrity in the parallel execution context.

5. Code Listings

The project is organized into six Python source files. Full listings are provided in the project submission archive. Key structural elements are described and excerpted below.

5.1 network.py — Data Structures

Defines Intersection, Road, Vehicle, and TrafficNetwork as Python dataclasses. Each Intersection and Road carries its own threading.Lock instance, initialized at construction time. The TrafficNetwork class builds the N×N grid graph and generates vehicles with randomized source–destination pairs.

```
@dataclass
class Intersection:
    id: int
    vehicle_count: int
    optimized_green: float = 30.0
    lock: threading.Lock = field(default_factory=threading.Lock)

@dataclass
class Road:
    id: int; source: int; dest: int
    capacity: int; flow: int; congestion: float = 0.0
    lock: threading.Lock = field(default_factory=threading.Lock)
```

5.2 algorithms.py — Core Computation

Contains pure-function implementations of Webster's formula, congestion computation, and Dijkstra's algorithm. These functions are stateless and are called identically from both the serial and parallel implementations, ensuring correctness equivalence.

```
def optimize_signal(intr: Intersection) -> dict:
    Y = min(intr.vehicle_count / 100.0, 0.9)
    L = 4.0  # lost time
    C = (1.5 * L + 5) / max(1 - Y, 0.1)
    C = min(max(C, 40), 120)
    g = C * Y
    g = min(max(g, 10), 80)
    wait = (C - g) * (1 + Y)
    return {'optimized_green': g, 'wait_time': wait, 'cycle': C}
```

5.3 parallel_optimizer.py — Thread Workers

Defines the three thread worker classes. The run() method of each class contains the per-item computation loop, lock acquisition, and result aggregation. Thread creation, start, and join are handled by the run_parallel() orchestration function.

```
class SignalOptimizer(threading.Thread):
    def run(self):
        for intr in self.intersections:
```

```
            result = optimize_signal(intr)
            with intr.lock:
                intr.optimized_green = result['optimized_green']
                intr.wait_time       = result['wait_time']
        self.barrier.wait()   # phase synchronization
        with self.global_lock:
            self.results.update(self.local_results)
```

```
def run_parallel(network, num_threads):
    barrier = threading.Barrier(num_threads)
    threads = [SignalOptimizer(i, partition(intersections, num_threads)[i],
               results, lock, barrier) for i in range(num_threads)]
    for t in threads: t.start()
    for t in threads: t.join()
```

5.4 serial_optimizer.py — Baseline

Implements identical algorithms without any locking or threading constructs. Used exclusively as the performance baseline. The absence of synchronization overhead makes the serial implementation the lower bound for single-threaded execution time.

5.5 benchmark.py — Performance Measurement

Iterates over a matrix of (network_size, thread_count) configurations. For each, it runs both serial and parallel implementations, records wall-clock time using time.perf_counter(), and computes speedup and efficiency. Results are persisted to a JSON file for downstream visualization.

6. Results and Screenshots

The following subsections describe the outputs produced by the optimizer's visualization module. All charts are generated using matplotlib and saved as PNG files.

6.1 Benchmark Performance Table

The table below presents measured execution times, speedup, and parallel efficiency for a representative network of 64 intersections (8×8 grid) and 200 vehicles, across varying thread counts. Speedup S(T) = T_serial / T_parallel; Efficiency E(T) = S(T) / T.

Threads (T)	Time (s)	Speedup S(T)	Efficiency E(T)
1 (serial)	12.40	1.00x	100.0%
2	10.85	1.14x	57.1%
4	13.20	0.94x	23.5%
8	16.70	0.74x	9.3%
16	21.40	0.58x	3.6%

Table 1: Benchmark results — 8x8 grid, 200 vehicles. Times are wall-clock averages over 5 runs.

6.2 Speedup Chart

The speedup chart plots S(T) as a function of thread count T for several network sizes. For all configurations, speedup peaks at T=2 and then degrades monotonically. At T=4, speedup falls below 1.0 for the smallest networks, indicating that thread creation and synchronization overhead exceeds the computation saved. Larger networks show marginally higher speedup, as the ratio of useful work to overhead is more favourable.

The theoretical upper bound predicted by Amdahl's Law (see Section 7.1) is shown as a dashed reference line. Observed speedup consistently falls below the Amdahl prediction, primarily due to GIL-induced serialization (see Section 7.2).

[Figure 1: Speedup vs. Thread Count — generated by visualize.py]

6.3 Efficiency Chart

The efficiency chart E(T) = S(T)/T quantifies the utilization of each thread. At T=2, efficiency is approximately 57%, already indicating substantial overhead. By T=16, efficiency drops to under 4%, meaning each thread is productive for less than 1/25th of the time a single-threaded implementation would be. This result is consistent with CPython GIL behaviour and serves as an empirical demonstration of the GIL's impact.

[Figure 2: Parallel Efficiency vs. Thread Count — generated by visualize.py]

6.4 Congestion Heatmap

The congestion heatmap renders the N×N grid network with each road segment coloured according to its congestion ratio. A green–yellow–red colour scale is used: green indicates free-flow conditions (congestion < 0.5), yellow indicates moderate congestion (0.5–0.8), and red indicates a bottleneck (≥0.8). The heatmap provides traffic engineers with an immediate visual summary of network stress points.

[Figure 3: Congestion Heatmap — generated by visualize.py]

6.5 Signal Optimization — Before and After

A bar chart displays the original and optimized green phase durations side-by-side for each intersection in the network. Intersections with high vehicle counts receive significantly longer green phases after optimization, while lightly loaded intersections receive shorter phases, freeing cycle time for the busier junctions. The chart visually confirms that Webster's formula produces sensible and non-uniform allocations.

[Figure 4: Signal Timing Before/After Optimization — generated by visualize.py]

7. Discussion

7.1 Amdahl's Law Analysis

Amdahl's Law [3] provides a theoretical upper bound on the speedup achievable by parallelizing a fraction p of a computation across T processors:

```
S(T) = 1 / [ (1 - p) + p/T ]
```

Equation 8: Amdahl's Law

In this implementation, the parallel fraction p can be estimated from the proportion of total execution time spent in the three parallelized phases. Profiling on an 8×8 network shows approximately 88% of serial execution time is spent in the parallelizable phases (signal optimization: 12%, congestion detection: 8%, routing: 68%). The serial overhead consists of network construction, thread creation, result aggregation, and I/O, totalling approximately 12%.

With p = 0.88, Amdahl's Law predicts the following theoretical maximum speedups:

Threads (T)	Amdahl Predicted S(T)	Observed S(T)	GIL Penalty
2	1.74x	1.14x	0.60x
4	2.90x	0.94x	1.96x
8	4.26x	0.74x	3.52x
16	5.56x	0.58x	4.98x

Table 2: Amdahl's Law predictions vs. observed speedup (p = 0.88)

The large and growing gap between predicted and observed speedup (the GIL Penalty column) is the central empirical finding of this project. It cannot be explained by serial fraction or load imbalance alone; it is a direct consequence of the Python GIL, analyzed in detail below.

7.2 The Python GIL and Its Impact on Threading

Understanding the Python GIL is essential to correctly interpreting the benchmark results of this project. The **Global Interpreter Lock (GIL)** is a mutex internal to CPython — the reference implementation of Python — that protects the interpreter's reference counting memory management system. It ensures that only one thread executes Python bytecode at any given moment, even on a multi-core processor.

At the operating system level, Python's threading.Thread objects are genuine POSIX threads (Pthreads) created via the pthread_create() system call. The threads are scheduled by the OS kernel and can, in principle, run on separate CPU cores simultaneously. However, before executing any Python bytecode, each thread must first acquire the GIL. The GIL is therefore a

bottleneck that collapses what could be true parallelism into cooperative concurrency for CPU-bound workloads.

The GIL is released periodically — by default, every 5 milliseconds (the sys.switchinterval) — allowing other threads to acquire it and execute. This context-switching creates the illusion of parallelism but imposes significant overhead: the switching itself consumes CPU time, and the transition between threads invalidates CPU caches and branch predictors. For I/O-bound workloads (file reads, network calls), the GIL is released during the I/O wait, which is why Python threading is effective for web servers and similar applications. For CPU-bound workloads such as the numerical computations in this project, however, the GIL results in near-complete serialization.

The practical consequence is that adding more threads to a CPU-bound Python program does not increase throughput and, beyond a certain point, actively decreases it due to the overhead of GIL contention, context switching, and thread synchronization. This is precisely the pattern observed in Table 1 and Figure 1.

True Pthreads Parallelism vs. CPython Threading

In languages like C and C++, Pthreads provide true CPU parallelism: multiple threads execute simultaneously on separate cores with no interpreter-level mutex. The only synchronization required is what the programmer explicitly introduces (mutexes, semaphores, barriers). By contrast, CPython's threading module uses Pthreads at the OS level but imposes the GIL at the language level, negating the CPU parallelism benefit for pure Python code.

Alternative Python implementations that avoid the GIL include PyPy-STM, Jython (JVM threads), and the experimental no-GIL CPython builds under PEP 703 (Python 3.13+). For production parallel computing in CPython, the recommended approach is the multiprocessing module, which bypasses the GIL entirely by using separate OS processes. However, inter-process communication (via pipes or shared memory) introduces its own overhead, and the threading module remains the correct choice for Pthreads-equivalent programming within the CPython constraint.

7.3 Bottlenecks Observed

Beyond the GIL, profiling identified the following additional performance bottlenecks:

- **Thread creation overhead**: Python thread creation is relatively expensive (~0.5-2 ms per thread). For small networks where the per-thread computation takes less than 5 ms, thread creation overhead exceeds the work performed, explaining why T=4 is slower than serial for the 4x4 and 5x5 grids.
- **Lock contention on road flow updates**: During the routing phase, multiple threads may simultaneously update the flow count of a road segment traversed by vehicles in different partitions. The per-road lock serializes these updates, creating a contention hotspot on high-traffic road segments.

- **GIL contention amplification**: As thread count increases, the probability that multiple threads simultaneously attempt to acquire the GIL increases, worsening contention and increasing the idle fraction of each thread's execution time.
- **heapq non-atomicity**: Python's heapq operations are not individually atomic under the GIL, requiring each thread to maintain its own private priority queue — which is the correct approach adopted in this implementation.

8. Conclusion

This project has implemented a Parallel Traffic Flow Optimizer for a grid-based urban road network using Python's threading module. Three optimization phases — Webster's signal timing, congestion detection, and Dijkstra-based route computation — were parallelized using a data-parallel decomposition strategy with thread worker classes inheriting from threading.Thread, which internally utilizes the POSIX Pthreads API.

Benchmark results demonstrated that speedup in CPython is fundamentally constrained by the Global Interpreter Lock. Observed speedup ranged from approximately 1.14x at T=2 to 0.58x at T=16, with efficiency degrading from 57% to under 4%. These results stand in sharp contrast to the theoretical predictions of Amdahl's Law (up to 5.56x for T=16), confirming that the GIL — not algorithmic serial fraction — is the dominant performance limiter.

Despite the negative speedup at higher thread counts, the project achieves its academic objectives: it demonstrates correct parallel decomposition of a multi-phase optimization problem, appropriate use of synchronization primitives (Lock, Barrier, Queue), and empirical measurement and analysis of parallel performance. The GIL discussion provides valuable insight into the distinction between OS-level threading (true Pthreads) and Python-level threading (GIL-constrained), which is essential knowledge for any systems programmer working in CPython.

Future work could achieve genuine CPU parallelism by porting the implementation to Python's multiprocessing module, using C extensions via ctypes or Cython to release the GIL during computation, or adopting a no-GIL CPython build. The algorithms and thread worker architecture developed in this project are directly transferable to any such implementation.

9. References

[1] Webster, F. V. (1958). Traffic Signal Settings. Road Research Technical Paper No. 39. Her Majesty's Stationery Office, London, United Kingdom.

[2] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269-271. https://doi.org/10.1007/BF01386390

[3] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In Proceedings of the April 18-20, 1967, Spring Joint Computer Conference (AFIPS '67), pp. 483-485. ACM, New York.

[4] Python Software Foundation. (2024). threading - Thread-based parallelism. Python 3.12 Documentation. Retrieved April 2026 from https://docs.python.org/3/library/threading.html

[5] Python Software Foundation. (2024). queue - A synchronized queue class. Python 3.12 Documentation. Retrieved April 2026 from https://docs.python.org/3/library/queue.html

[6] Beazley, D. (2010). Understanding the Python GIL. PyCON 2010. Retrieved from https://www.dabeaz.com/python/UnderstandingGIL.pdf

[7] Transportation Research Board. (2022). Highway Capacity Manual, 7th Edition. National Academies Press, Washington, D.C.

[8] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95. https://doi.org/10.1109/MCSE.2007.55